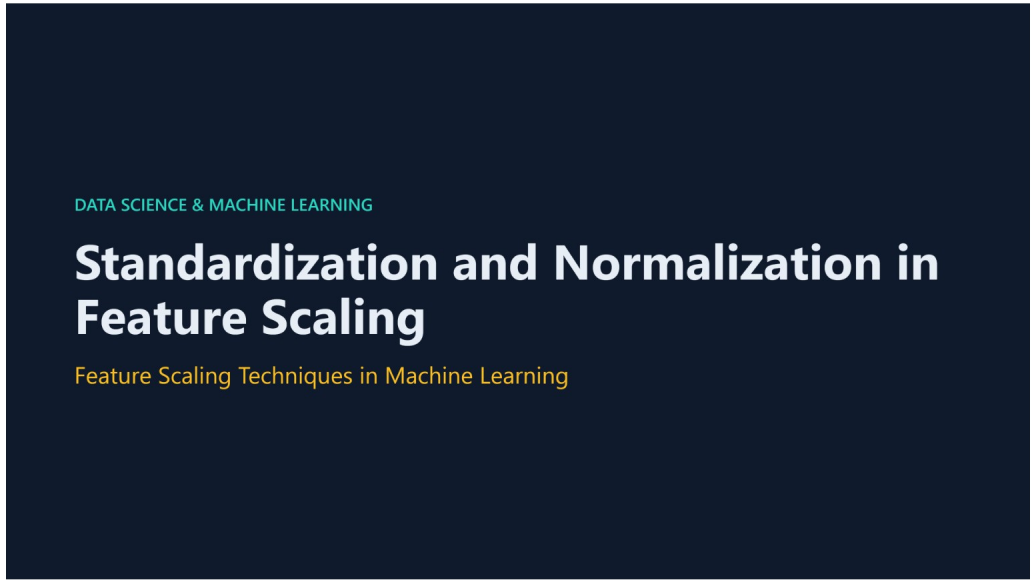
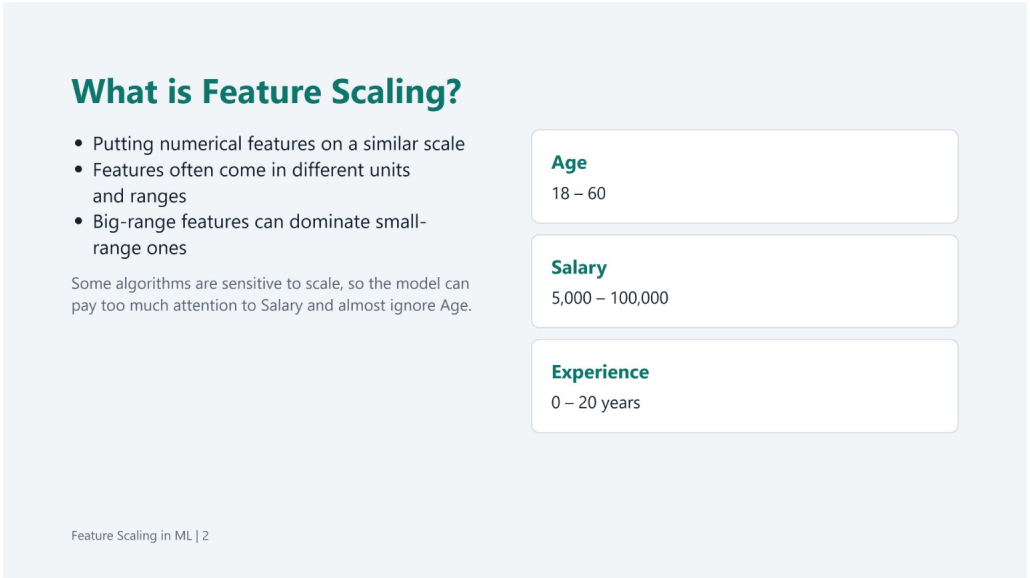
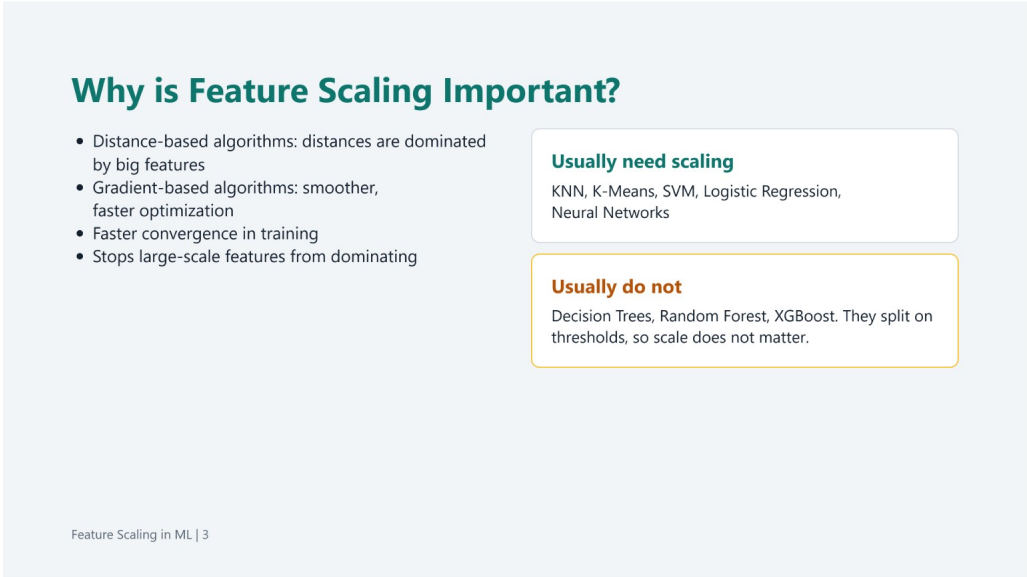


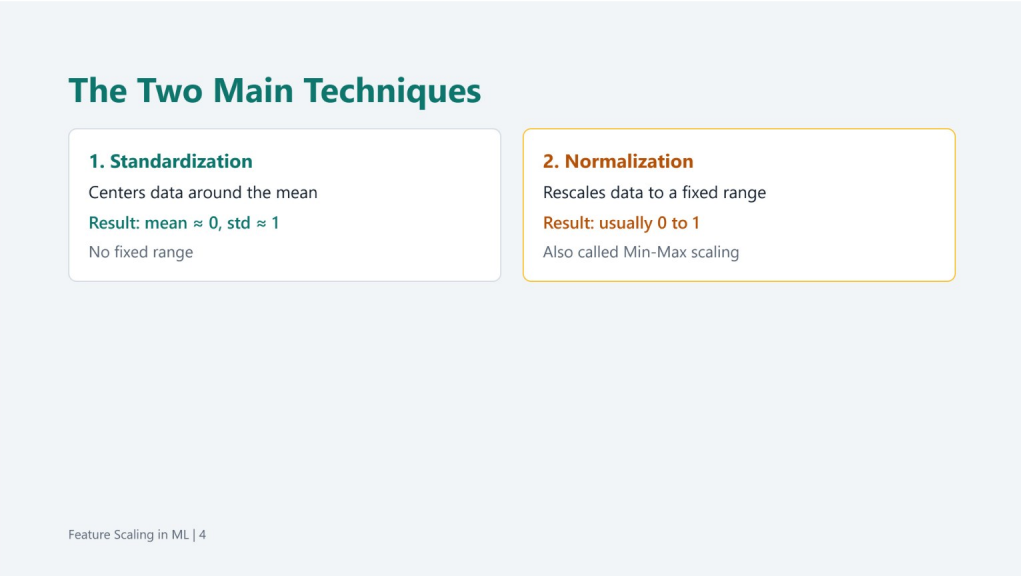
Hello everyone. Today I will explain feature scaling, and mainly two techniques: standardization and normalization. I will keep it simple and practical, with small examples and some Python code.



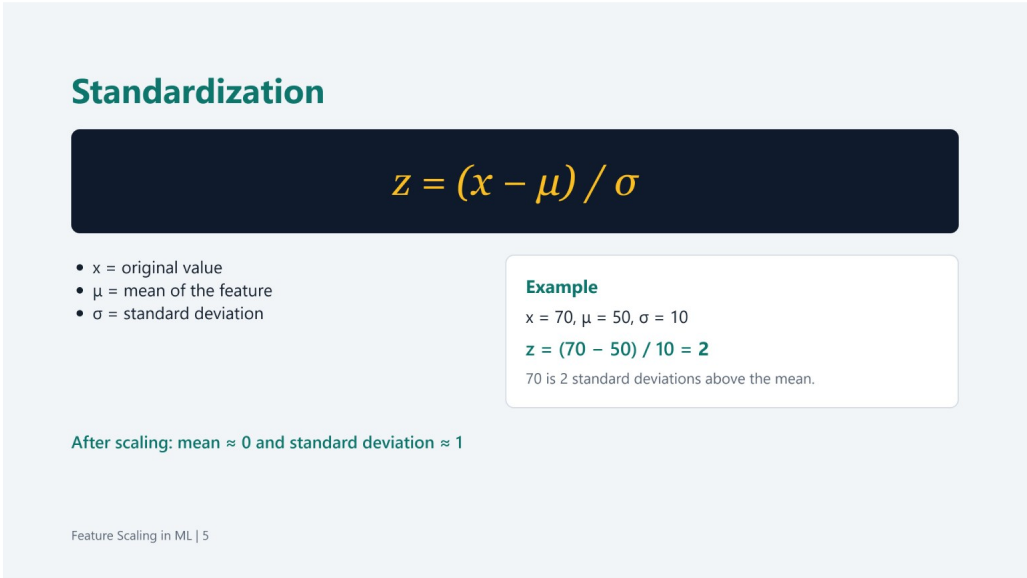
Feature scaling means changing numerical features so they are on a similar scale. In real data, features have different units. Look at this example: Age goes from 18 to 60, Salary from 5,000 to 100,000, and Experience from 0 to 20. Salary is thousands of times bigger in range. For some algorithms, this means Salary will dominate the result just because its numbers are large, not because it is more important.



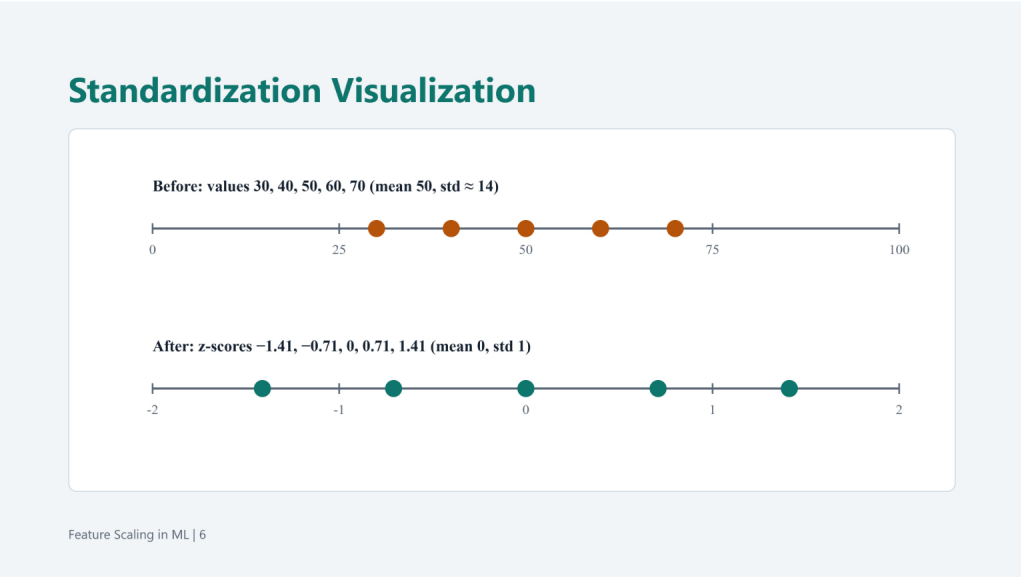
Scaling is important for two main groups. Distance-based algorithms, like KNN and K-Means, use distances, so a big feature controls the distance. Gradient-based models, like logistic regression and neural networks, train faster and more stably when features have similar scales. But scaling is not always needed. Tree-based models such as decision trees, random forest, and XGBoost split data using thresholds, so they usually do not need it.



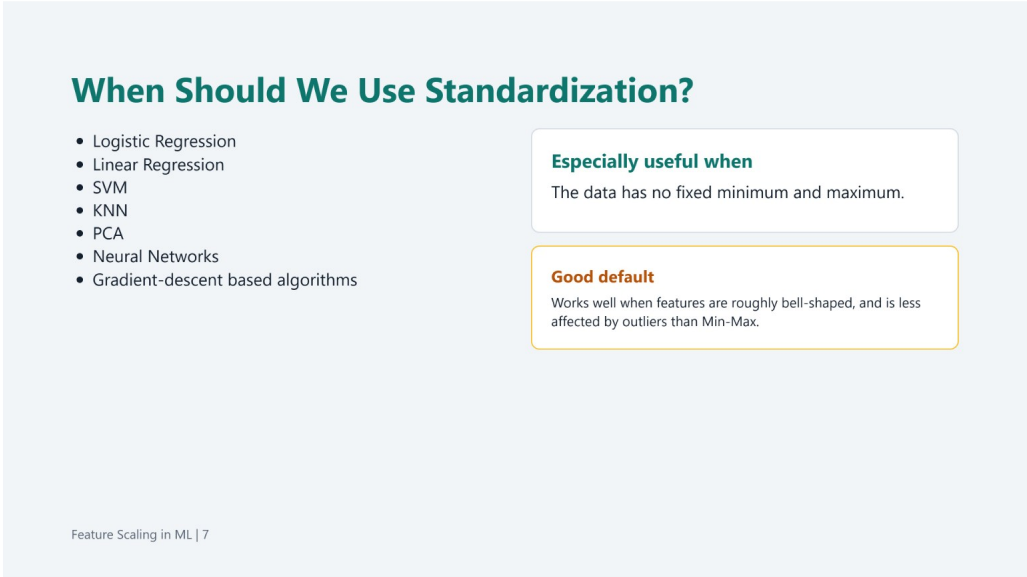
There are two main techniques. Standardization centers the data around the mean, so the mean becomes about zero and the standard deviation about one. Normalization, or Min-Max scaling, squeezes the values into a fixed range, usually zero to one. Next, we will see each one in detail.



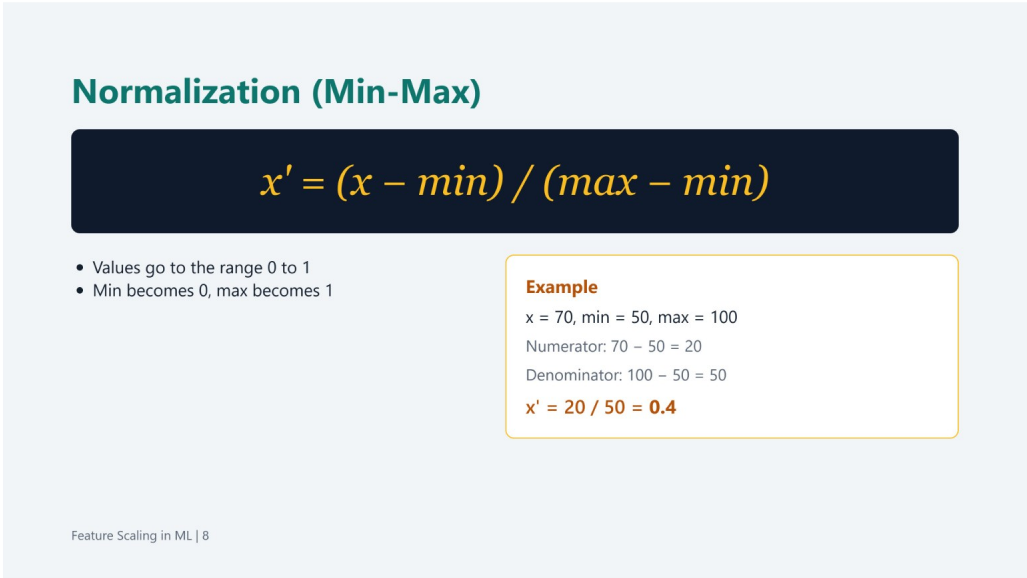
Standardization uses the formula z equals x minus mean, divided by the standard deviation. After this, the feature has mean about zero and standard deviation about one. Example: x is 70, mean is 50, standard deviation is 10. So z equals 20 divided by 10, which is 2. It means this value is two standard deviations above the mean.



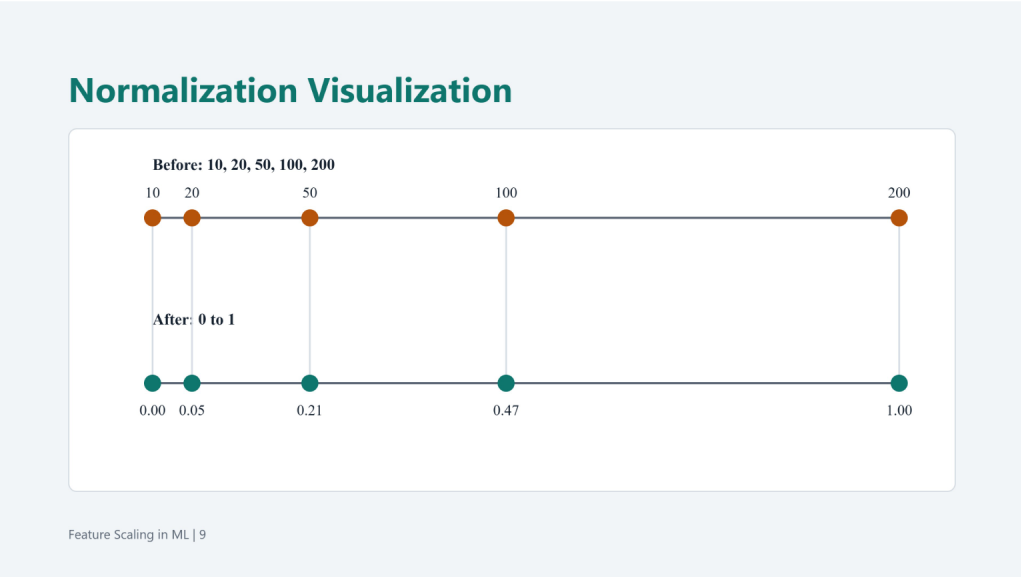
Here is a simple picture. Before standardization, the values are spread over a big range, like 30 to 70 around a mean of 50. After standardization, the same points are centered around zero, and the spread is about one. The shape of the data stays the same. Only the scale and the center change.



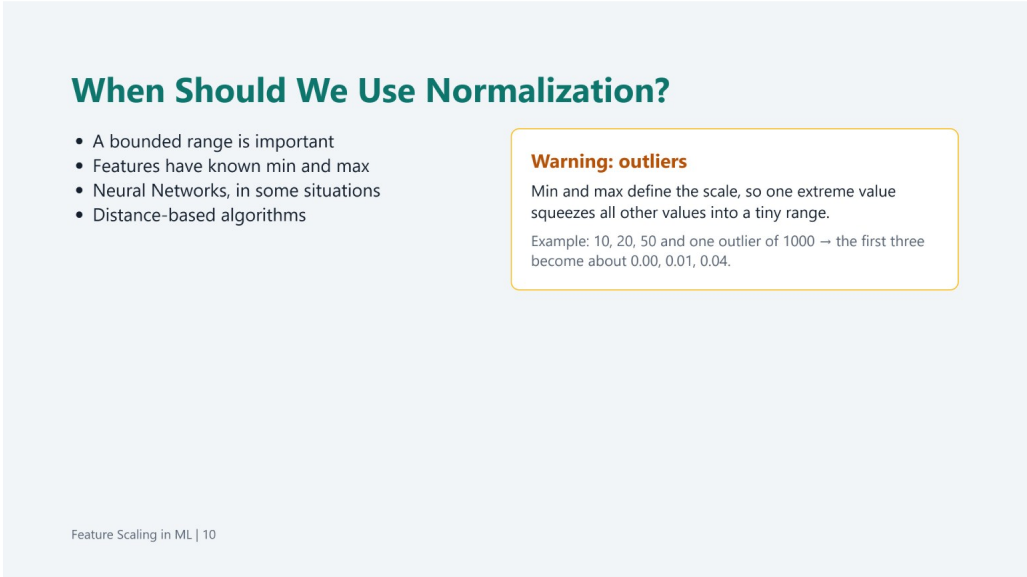
Standardization is widely used. It is common for logistic and linear regression, SVM, KNN, PCA, neural networks, and any algorithm that uses gradient descent. It is especially useful when the data has no fixed minimum and maximum. For PCA, it is very important, because PCA looks at variance, and big features would dominate.



Min-Max normalization uses x minus the minimum, divided by maximum minus minimum. The smallest value becomes zero and the largest becomes one. Example: x is 70, min is 50, max is 100. The top part is 70 minus 50, which is 20. The bottom part is 100 minus 50, which is 50. So the result is 20 over 50, which equals 0.4.



Now the same idea with Min-Max. We have the values 10, 20, 50, 100, and 200. The minimum is 10 and the maximum is 200. After normalization, 10 becomes 0, 200 becomes 1, and the others fall in between: 0.05, 0.21, and 0.47. The order and the relative distances are kept, but all values are now between 0 and 1.



Use Min-Max normalization when you need a bounded range, or when you know the real minimum and maximum, like pixel values from 0 to 255. It is also used with neural networks in some cases, and with distance-based algorithms. But be careful: it is sensitive to outliers. One very large value changes the max, and all normal values get squeezed into a small range near zero.

Standardization vs Normalization

Feature	Standardization	Normalization
Main idea	Center around mean	Scale to fixed range
Formula	(x − μ) / σ	(x − min) / (max − min)
Typical range	No fixed range	Usually 0–1
Mean	≈ 0	Not necessarily 0
Std	≈ 1	Not necessarily 1
Sensitive to outliers	Less sensitive than Min-Max	More sensitive
Common tools	StandardScaler	MinMaxScaler

This table summarizes the differences. Standardization centers around the mean, and normalization scales to a fixed range. Standardization gives mean about 0 and std about 1, and has no fixed range. Normalization usually gives 0 to 1, but the mean and std are not fixed. Min-Max is more sensitive to outliers. In scikit-learn we use StandardScaler and MinMaxScaler.

Example with a Small Dataset

Age	Salary	Experience
20	5,000	1
25	10,000	3
30	30,000	5
40	60,000	10
50	100,000	20



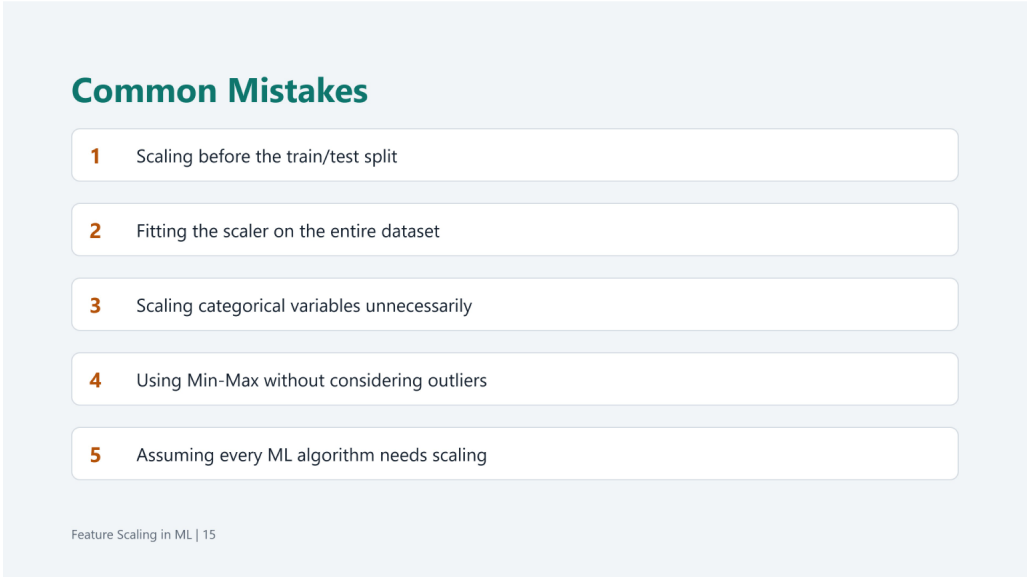
Here is a small dataset with Age, Salary, and Experience. Age ranges over 30 units and experience over 19, but salary ranges over 95,000. If we use KNN, the distance will depend almost only on salary. So the model will ignore age and experience. After scaling, all three features have a fair influence.



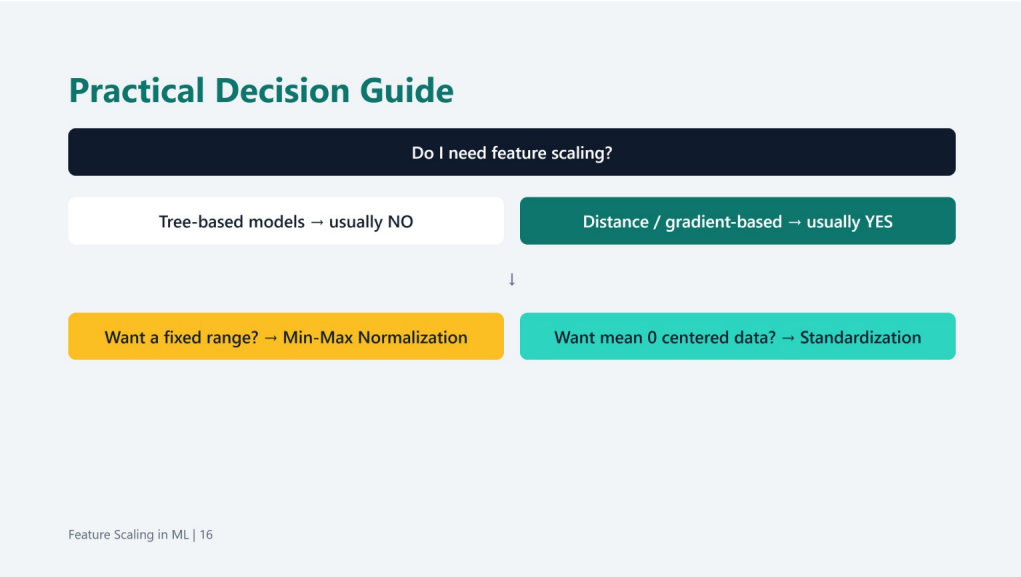
The code is short. First, we import StandardScaler and MinMaxScaler from scikit-learn. Then we create a scaler object. The method fit_transform does two things: fit learns the values from the data, like the mean and standard deviation, or the min and max. Then transform applies the formula. Same steps for both scalers. Only the class changes.



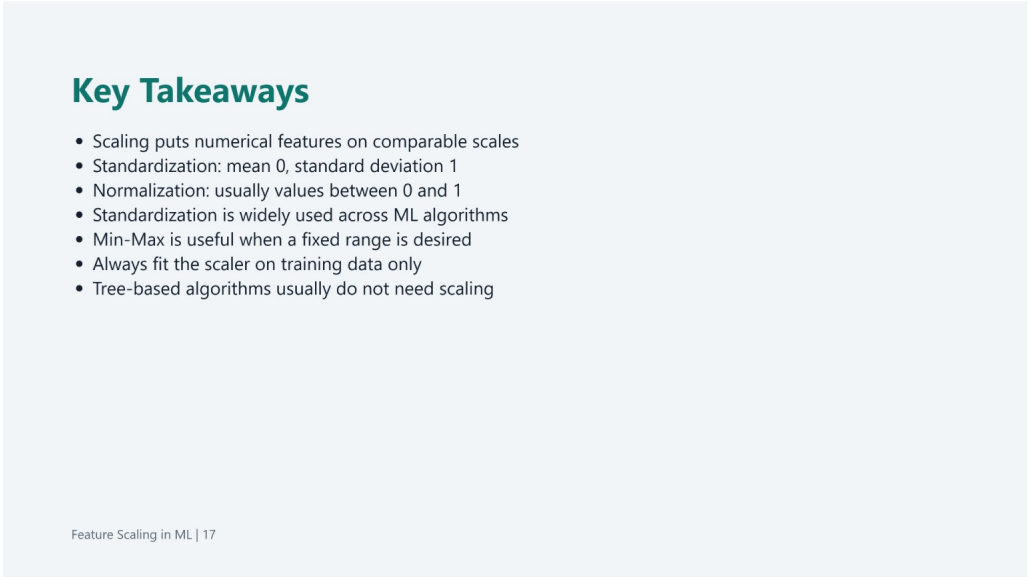
This is a very important rule. We fit the scaler only on the training data. Then we use the same scaler to transform both train and test. We should not fit it again on the test set, and not on the full dataset. Otherwise the mean, std, min, or max from the test data influence the training. This is called data leakage, and it gives results that are too optimistic.



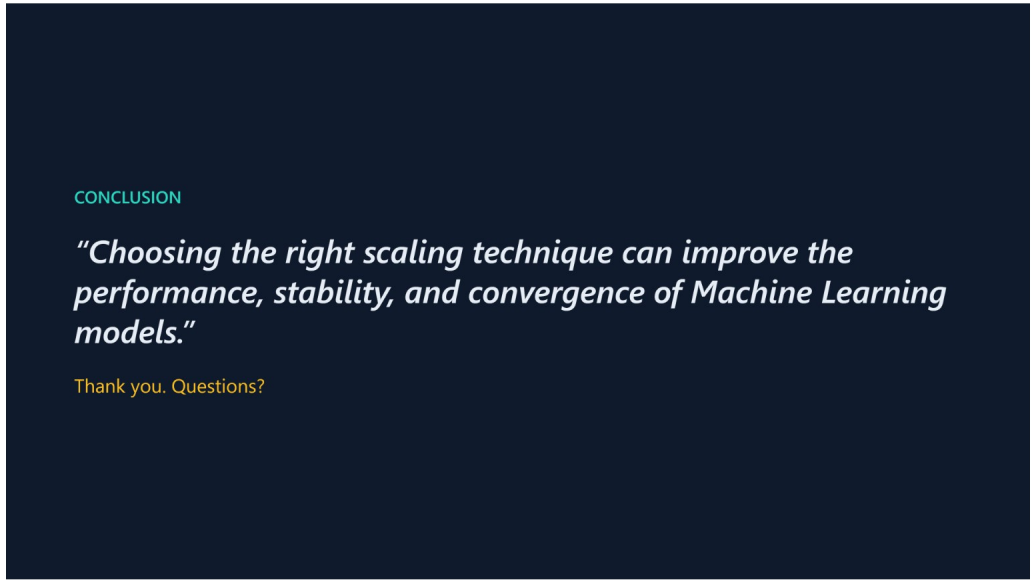
Here are five common mistakes. One, scaling before the train-test split. Two, fitting the scaler on the whole dataset. Both cause data leakage. Three, scaling categorical variables, like one-hot columns, when it is not needed. Four, using Min-Max when you have strong outliers. And five, thinking every algorithm needs scaling. Tree-based models usually do not.



Use this simple guide. First ask: do I need scaling? If I use tree-based models, usually no. If I use distance-based or gradient-based models, usually yes. Then choose the technique. If I want a fixed range, I use Min-Max normalization. If I want centered data with mean zero, I use standardization. When in doubt, standardization is a safe default.



To summarize. Scaling puts features on comparable scales. Standardization gives mean zero and std one. Normalization usually gives values from zero to one. Standardization is widely used, and Min-Max is good when we want a fixed range. The most important rule is to fit the scaler on the training data only. And tree-based models usually do not need scaling.



To finish: choosing the right scaling technique can improve the performance, stability, and convergence of machine learning models. Thank you for listening. I am happy to answer any questions.